APL Quantum

REVIEW ARTICLE | MARCH 01 2024

Open hardware solutions in quantum technology

Nathan Shammah ; Anurag Saha Roy ; Carmen G. Almudever; Sébastien Bourdeauducq ; Anastasiia Butko; Gustavo Cancelo ; Susan M. Clark; Johannes Heinsoo ; Loïc Henriet; Gang Huang ; Christophe Jurczak ; Janne Kotilahti; Alessandro Landra ; Ryan LaRose; Andrea Mari; Kasra Nowrouzi; Caspar Ockeloen-Korppi; Guen Prawiroatmodjo; Irfan Siddiqi ; William J. Zeng

Check for updates

APL Quantum 1, 011501 (2024)
https://doi.org/10.1063/5.0180987

View Online
Export Citation

Articles You May Be Interested In

p-bits for probabilistic spin logic

Appl. Phys. Rev. (March 2019)

Hardware for dynamic quantum computing

Rev. Sci. Instrum. (October 2017)

The EOSDIS software challenge

AIP Conf. Proc. (August 1993)

04 September 2026 20:03:04

AIP Publishing

Special Topics Open for Submissions

Learn More

AIP Publishing

APL Quantum REVIEW pubs.aip.org/aip/apq

Open hardware solutions in quantum technology

Cite as: APL Quantum 1, 011501 (2024); doi: 10.1063/5.0180987
Submitted: 13 October 2023 • Accepted: 3 January 2024 •
Published Online: 23 February 2024

View Online Export Citation CrossMark

Nathan Shammah,[1a)] Anurag Saha Roy,[2] Carmen G. Almudever,[3] Sébastien Bourdeauducq,[4] Anastasiia Butko,[5] Gustavo Cancelo,[6] Susan M. Clark,[7] Johannes Heinsoo,[8] Loïc Henriet,[9] Gang Huang,[10] Christophe Jurczak,[11] Janne Kotilahti,[8] Alessandro Landra,[8] Ryan LaRose,[12] Andrea Mari,[1,13] Kasra Nowrouzi,[5] Caspar Ockeloen-Korppi,[8] Guen Prawiroatmodjo,[14] Irfan Siddiqi,[5,15] and William J. Zeng[1,11]

AFFILIATIONS

[1]Unitary Fund, San Francisco, California 94104, USA
[2]Qruise GmbH, 66113 Saarbruecken, Germany
[3]Technical University of Valencia, Valencia, Spain
[4]M-Labs Limited, North Point, Hong Kong
[5]Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, Berkeley, California 94720, USA
[6]Fermi National Accelerator Laboratory, Batavia, Illinois 60510, USA
[7]Sandia National Laboratories, Albuquerque, New Mexico 87123, USA
[8]IQM Quantum Computers, Espoo 02150, Finland
[9]PASQAL, 7 rue Leonard de Vinci, 91300 Massy, France
[10]Accelerator Technology and Applied Physics Division, Lawrence Berkeley National Laboratory, Berkeley, California 94720, USA
[11]Quantonation, 75010 Paris, France
[12]Center for Quantum Computing, Science, and Engineering, Michigan State University, East Lansing, Michigan 48824, USA
[13]School of Science and Technology, Università di Camerino, 62032 Camerino, Italy
[14]Microsoft Quantum, Redmond, Washington 98052, USA
[15]Quantum Nanoelectronics Laboratory, Department of Physics, University of California at Berkeley, Berkeley, California 94720, USA

[a)]**Author to whom correspondence should be addressed:** nathan@unitary.fund

ABSTRACT

Quantum technologies, such as communication, computing, and sensing, offer vast opportunities for advanced research and development. While an open-source ethos currently exists within some quantum technologies, especially in quantum computer programming, we argue that there are additional advantages in developing open quantum hardware (OQH). Open quantum hardware encompasses open-source software for the control of quantum devices in labs, blueprints, and open-source toolkits for chip design and other hardware components, as well as openly accessible testbeds and facilities that allow cloud-access to a wider scientific community. We provide an overview of current projects in the OQH ecosystem, identify gaps, and make recommendations on how to close them at present. More open quantum hardware would accelerate technology transfer to and growth of the quantum industry and increase accessibility in science.

© 2024 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution (CC BY) license (http://creativecommons.org/licenses/by/4.0/). https://doi.org/10.1063/5.0180987

04 September 2026 20:03:04

I. INTRODUCTION

The free and open exchange of scientific tools has become more and more important as automation and devices increase their role in scientific fields. The past five years have witnessed an explosion of open-source tools for programming quantum computers. The open nature of these software tools has substantially increased the number of users of quantum computers and created a new genre of programmer: the "quantum software engineer."[1–6] This has shaped the development of quantum computing as a whole, leading to the creation of new organizations, job categories, and career paths.

Less attention has been paid to the tools and components actually used to build and control quantum computers—and quantum technologies such as communications and sensing more broadly—as well as efforts to make quantum computers more accessible in non-commercial ways. We use the term "open quantum hardware" (OQH) to cover them, and intend for it to explicitly encompass several steps related to the openness associated with hardware in quantum technology, including (1) open-source software (OSS) for designing quantum processors and other hardware components (used for computation, but also for simulation, sensing, and communication), (2) foundries and facilities for fabricating quantum processors, (3) software for controlling, analyzing and characterizing quantum devices, and (4) software and hardware for making the infrastructure that enables various levels of open-access—from cloud-accessible quantum processors to collaborative testbeds providing non-commercial access and testing, from remotely accessible research labs to the related cloud infrastructure. These steps covering the life cycle of OQH and their overlapping relations are sketched in Fig. 1.

In this article, we provide an overview of efforts in each of these categories of open quantum hardware (OQH), highlight notable projects in each category, identify gaps, and provide recommendations for closing them. A major theme in this review is that there is much opportunity to promote interoperability, reduce costs, and increase the number of users of quantum technologies.

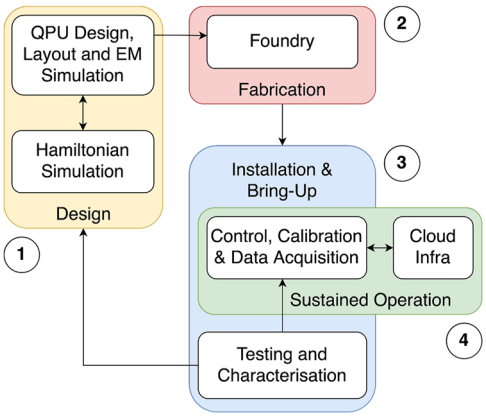


FIG. 1. Overarching diagram of the open quantum hardware steps and their interconnection. 1: Design phase, which can involve a loop between the Hamiltonian and electromagnetic (EM) simulations used to define the QPU design and layout. 2: Fabrication step, e.g., through a foundry. 3: Installation and bring-up, which includes testing and characterization; information collected at this step can be fed back to step 1 to modify designs. 4: Sustained operation, with overlaps (with step 3), is composed of data acquisition (with control and calibration) and can involve interfacing with infrastructure to provide cloud access to the device or experiment. While these steps also appear in more general quantum hardware projects that are not open, their separation and standardization are required features to interconnect open frameworks.

Enabling the open quantum hardware ecosystem is both the natural complement to the open quantum software ecosystem (which is itself quite robust and sophisticated[2]) as well as the natural extension of efforts to open up classical hardware. This ecosystem also represents a prime opportunity for entities building hardware (of all kinds) to engage in the kind of beneficial, pre-competitive activity that boosts the ecosystem as a whole (and consequently, those builders themselves).

A first generation of projects in open quantum hardware, such as ARTIQ and pyEPR,[7,8] has pioneered the free and open dissemination of tools. We are now witnessing the beginning of a more robust and sophisticated OQH ecosystem,[9,10] which can enable and accelerate scientific progress and discovery on a wider scale. On the one hand, this ecosystem can further extend beyond quantum computing, on which it is mostly focused, to encompass quantum technologies such as quantum sensing and communications. On the other hand, it can further integrate and benefit from the adoption and integration of existing non-quantum open hardware projects and frameworks.[11] What is more, given the substantial investments made around the world[12]—especially in Australia,[13] Eurasia,[14–19] and North America[20–23]—to support the development of a quantum technology industry, ensuring those investments yield the best possible fruit is of great importance. One way to do so is by encouraging the development of open communities and ecosystems[24–33] around quantum technology.

To describe the potential of OQH, we can look at the open quantum software ecosystem, which has flourished within the past five years, thanks to seeds planted over ten years ago. As a result, researchers (theorists and experimentalists alike) as well as quantum software engineers worldwide can use open-source software to advance quantum technology and quantum science in many directions, generally building upon an existing stack. An illustrative case is that of the Quantum Toolbox in Python (QuTiP),[34,35] first released in 2012, which enables the exploration of the effects of noise on a variety of quantum systems interacting with the environment. Building on top of QuTiP, several other tools have emerged, focusing on specific niches such as nontrivial system-environment dynamics,[36,37] notably getting closer and closer to the simulation of quantum processing units (QPUs) and their diagnostics.[38,39]

When looking at similar efforts in the classical open hardware space, the astounding success of the Arduino project[40] (microcontrollers) as well as the Raspberry Pi[41] (single-board computers) speaks to the power of putting open hardware in the hands of end-users. We can reasonably expect similar benefits within the quantum hardware ecosystem. Compared to only open-source software, an open hardware ecosystem also provides the legibility, transparency, and reproducibility of hardware devices, a crucial consideration for supply chains and their management.

Finally, the pre-competitive activity that can take place by opening up quantum hardware extends these benefits by mobilizing institutional momentum from key players who can help the ecosystem adopt a broad ethos of openness, interchangeability, interoperability, and benchmarking, as fostered by bodies like the Quantum Economic Development Consortium (QED-C) in the USA and

04 September 2026 20:03:04

© Author(s) 2024

TABLE I. OQH examples grouped by functionality. The different functionalities in open quantum hardware and examples of representative projects: processor design, Sec. II A; simulation and diagnostics, Sec. II B; and control and data acquisition, Sec. II B 4. We also list programs related to facilities in OQH: remotely accessible labs (where we exclude here the many commercial providers and cloud services that are discussed in Sec. II D 1); testing (testbeds), Sec. II D 2; and fabrication (foundries), Sec. II D 3.

	Functionality	Examples
Projects	Processor design	DASQA,[45,46] KQCircuits,[47] PainterQubits/Devices.jl, pyEPR,[8] Qiskit Metal,[48] QuCAT[49]
	Simulation and diagnostics	KQCircuits,[47] Pulser,[50] Qiskit Metal,[48] QuTiP,[34,35] QuTiP-QIP,[39] sc-qubits,[38] Strawberry Fields[51]
	Control and data acquisition	ARTIQ,[7] Duke-ARTIQ,[52] Qua[a],[53] QCoDeS,[54] QICK,[55] Quantify,[56] QubiC,[57] Qudi[58] qupulse,[59] Sinara Open Hardware[60]
Facilities	Remotely accessible labs[b]	Forschungszentrum Jülich through OpenSuperQ,[61] Quantum Inspire[62]
	Testing (testbeds)	Lawrence Berkeley National Lab's AQT, Open Quantum Design, Sandia National Labs' QSCOUT,[63] Sherbrooke's Distriq DevTeQ, NQCC[64]
	Fabrication (foundries)	LPS qubit Collaboratory, UCSB quantum foundry, QuantWare[c]

[a]Partially open-source.
[b]Excluding commercial providers.
[c]Private company with support for Qiskit Metal.

similar entities in other geographies.[42–44] In doing so, this allows for additional adoption and use of quantum hardware in academic labs and testbeds (a point returned to in Sec. II D 2).

This article is organized as follows: Sec. II gives a high-level overview of the state of the art in OQH, divided among blueprints and software for hardware design (Sec. II A) and software for control and data acquisition (Sec. II B), itself divided into data acquisition (Sec. II B 1), pulse-level control on hardware (Sec. II B 2), pulse-level simulation (Sec. II B 3), and optimal control, calibration, and characterization (Sec. II B 4). In Sec. II C, we provide an outlook on the current status and specific need for OQH for quantum error correction. In Sec. II D, we review facilities for OQH, such as remotely accessible labs (Sec. II D 1), testbeds (Sec. II D 2), and foundries (Sec. II D 3). Throughout Sec. II subsections, we focus on some example projects to bring definiteness to the discussion. In Sec. III, we discuss gaps in OQH and make recommendations to close them. Finally, in Sec. IV, we give our conclusions.

II. OPEN QUANTUM HARDWARE AT PRESENT

Open quantum hardware encompasses open-source software that is used for designing, analyzing, building, controlling, and programming quantum chips, foundries that build quantum chips, and cloud-accessible labs and testbeds that provide alternative, non-commercially driven access to them. In this section, we review the state of open quantum hardware in each of these categories. It should be noted that within each of these categories, there can exist multiple stacks—configurations of components that are all interoperable with one another but not necessarily with components used in a different stack. Ensuring the interoperability of components across stacks

TABLE II. OQH examples grouped by architecture. The different open hardware projects, grouped by architectures (atoms, ions, photonics, spins, and SC qubits), in the bottom row include projects that are architecture-agnostics. Italics are used to highlight programs at facilities (e.g., instead of software projects).

	Atoms	Ions	Photonics	Spins	SC qubits
Projects	Bloqade.jl, Labscript-qc, Pulser, QICK	ARTIQ	Perceval, Strawberry Fields	QICK, Qudi, Qupulse	ARTIQ, DASQA, KQCircuits, PyEPR, QICK, Qiskit Metal, QubiC, Sc-qubits
Facilities	*AQT*, *Open-SuperQ*	*Open Quantum Design*, *QSCOUT*		*LPS Collab-oratory*	*LPS Collaboratory*, *Quantum Inspire*
Architecture-agnostic	QCoDeS, Qibolab, Qua, Quantify, QuOCS, QuTiP-QIP, *QCUP*				

04 September 2026 20:03:04

© Author(s) 2024

would help ensure a more robust and sophisticated ecosystem. In Table I, we summarize the different categories, providing examples of existing open hardware projects and facilities. We group them by functionalities: different open-source and open-hardware projects are present in processor design, simulation and diagnostics, control, and data acquisition. We also group facilities and organizations: remotely accessible labs, testbeds, and foundries.

In Table II, we highlight how different projects and facilities (emphasized in italics in the table) are distributed on the main quantum technology architectures: atoms, ions, photonics, spins, and SC qubits. It is visible that some architectures are populated with more projects, such as SC qubits and atoms. On the contrary, just a few tools exist for ions, with notable exceptions provided by ARTIQ and the Open Quantum Design Foundation. Tools that have strong cross-platform adoption are shown in the last row of Table II. In Secs. II A–II D, we explore in greater depth how different tools have been developed for different functionalities and are used in various architectures.

A. Blueprints and software for hardware design

Blueprints for hardware design are classic examples of "open hardware."[46,65–73] In modern device design, software tools have been developed to tackle various aspects of this task, known as computer-aided design (CAD). In quantum computing research, while quantum device design is often published in peer-reviewed papers and micrographs are included in the supporting materials and figures, generally, CAD drawings are not shared in the open. Notable exceptions include pyEPR,[8] Qiskit Metal,[48] and KQCircuits,[47] three projects enabling various capabilities for chip design of superconducting-circuit (SC) qubits.

Superconducting circuits incorporating non-linear devices, such as Josephson junctions and nanowires, are among the leading platforms for emerging quantum technologies. Using pyEPR, one can design and optimize SC circuits and control dissipative Hamiltonian parameters with a systematic, simple, and robust approach. This reduces the number of required *ab initio* simulations. pyEPR has been used on a variety of circuit quantum electrodynamics (cQED) devices and architectures, from 2D to 3D, including "2.5D" (flip-chip), demonstrating 1%–10% agreement for non-linear coupling and modal Hamiltonian parameters over five orders of magnitude and across a dozen samples.

Finite Element Methods (FEM) simulations can be obtained with PyEPR using the energy participation ratio approach (EPR). EPR unifies the design of dissipation and Hamiltonians around a single concept—the energy participation, a number between zero and one—in a single-step electromagnetic simulation. After the FEM simulations have validated the qubit properties (and possibly its coupling to a cavity), one is ready to fabricate the SC QPU. To this end, a photomask is generated to place the SC qubits on the substrate of a processor. You can find a typical CAD drawing of a superconducting flip-chip QPU and an example mask generated by KQCircuits[47] in Fig. 2, and more information about KQCircuits is in the frame below (*Example 1*).

> *Example 1, Hardware Design:* **"KQCircuits: KLayout Python library for integrated quantum circuit design."** KQCircuits[47] is an open-source Python library created by IQM, a full-stack quantum computing startup, for designing superconducting quantum circuits. It automates the layout and simulation part of the design process, outputting chip layouts and photomask files in OASIS or GDSII formats, standard formats for the specification of data structures for photomasks, and integrated circuit layout. The layout files are sent to a mask manufacturer to produce a physical mask, which is then used for the fabrication process. As a part of the mask export process, KQCircuits also produces other files such as netlists to help with design verification. While KQCircuits itself cannot be used to perform simulations, it can be used to export automated simulation scripts and files for popular electromagnetic field simulation toolkits, such as Ansys HFSS/Q3D,[74] pyEPR[8] (Energy Participation Ratio

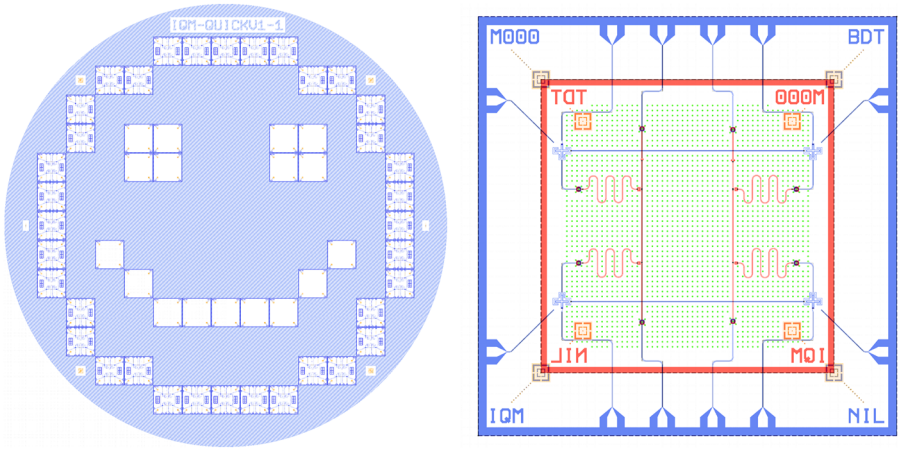


FIG. 2. Photomask layout and chip design. An open-source photomask (left) provided in KQCircuits as a demo example. An individual SC quantum processor layout (right) based on flip chip technology, where blue is one substrate, red is the other substrate, and green denotes the bump bonds. The chip contains four transmons capacitively coupled and two buses for multiplexed readout. The code for the mask can be accessed here, and the code for the chip can be accessed here.

04 September 2026 20:03:04

© Author(s) 2024

TABLE III. Open hardware design of SC qubit processors. Qiskit Metal, supported by IBM, and KQCircuits, developed at IQM, are two software packages for open hardware design for SC qubit processors. We highlight their purpose, integration with FEM simulation software, circuit analysis characteristics, and input and output file formats. EPR: Energy Participation Ratio, GDS: Graphic Design System, HFSS: High-Frequency Structure Simulator, LOM: Lumped Oscillator Model.

Software	Purpose	Simulations	Circuit analysis	Input/output (I/O)
Qiskit Metal	Full-stack quantum processor design	Ansys HFSS/Q3D, Ansys, GDS, etc. by plugins. Work in progress on open-source EM renders	EPR, impedance, quasi-lumped LOM, lumped	Python based. Connects to Ansys, GDS, etc. by plugins. Work in progress on open-source EM renders
KQCircuits	Design parametrization and automation for quantum processors. Extensive components, chips, and masks library	Ansys HFSS/Q3D, Sonnet, and Elmer (open-source) supported	EPR, cross-sectional EM properties, London equations, and netlist export	Import/export GDSII and OASIS format. Netlist export. Automated simulation scripts and files export

simulation), Sonnet,[75] and Elmer.[76] These simulations use the parameterized geometry of the physical mask layouts produced by KQCircuits. A comparison between KQCircuits and Qiskit Metal is provided in Table III.

KQCircuits generates multi-layer two-dimensional geometry, representing common structures in quantum processing units (QPU). An important part of KQCircuits are the definitions of parameterized geometrical objects, or "elements," and a framework for easily defining new ones. Generating the designs using code makes it easy to quickly create different variations of designs and helps to avoid costly human errors. The combination of many elements into a full QPU design is made easier by features such as named reference points used for automatic positioning of elements relative to each other and Graphical User Interface (GUI) editing. Furthermore, KQCircuits includes a library of premade chips, many of which have been manufactured and used for testing at IQM.

KQCircuits works on top of KLayout,[11] an open-source layout design software that is mainly used for classical electronics. Advantageously, existing KLayout functionalities can be used for quantum hardware other than classical, while KQCircuits only need to add features specific to QPU design. This connection with KLayout can also help to bring together the wider electronics open-source hardware community and the open quantum hardware community.

In terms of remaining challenges for chip design tools, we note that computational electromagnetic simulation of quantum circuits is still heavily reliant on the use of proprietary and expensive tools such as Ansys HFSS or Sonnet or CST Studio (full-wave and capacitance simulations in particular). Open-source tools such as openEMS or MEEP or Scuff-EM, which are used elsewhere in the field of microwave simulations, find very little adoption and application in quantum device design. Notable open-source exceptions are Elmer[76] and Palace.[77] Elmer is an open-source parallel multi-physics Finite Element Methods (FEM) software used for quantum device simulation on desktop and High Performance Computing (HPC), e.g., 3D layout capacitance matrices, the cross-section of layouts, and London equations. In the framework of OpenSuperQPlus (the European Open-Access Quantum Computer Project[61]), it was developed in partnership with CSC and IQM. Elmer, along with Gmsh,[78] has also been integrated as a backend for mesh generation and FEM simulation in Qiskit Metal. Palace (Parallel, Large-scale Computational Electromagnetics)[77] is an open-source parallel FEM software capable of full-wave electromagnetics simulations developed at AWS with out-of-the-box support for the use of large-scale cloud HPC resources.

B. Control and data acquisition

In this section, we discuss the features and components involved in the execution and readout of quantum experiments. This field is not easily defined, as the operation of a QPU or quantum technology experiment is already informed at the highest level with the definition of abstract instructions, e.g., for quantum programming, gates in the form of a quantum circuit in a high-level software development kit (SDK). To narrow the field, we focus here on control that gets "closer to the metal," e.g., lower than gate-level quantum circuit compilation in quantum computing. This field is already populated by a number of projects, spanning from pulse-level simulation of devices for diagnostics, characterization, and calibration to pulse-level control and data acquisition during experiment runs.[7,34,38,39,54,55,79–95] We highlight that core to these tasks is the exchange of data through application programming interfaces (APIs).

1. Data acquisition

Quantum hardware for control and readout of quantum systems includes a user interface to operate the instrument, for instance, through a touch-based front panel or remotely accessible GUI. However, to implement more complex tasks such as characterization,

04 September 2026 20:03:04

© Author(s) 2024

tuning, calibration, and general operation of QPU elements that involve coordination and timing between several different pieces of programmable hardware, it is essential to add a software layer that runs on a control computer and communicates with hardware directly through a programming interface. This software layer orchestrates the flow of control and data acquisition commands and is responsible for the overall operation of the QPU via the quantum hardware.

On the software side, there exist quite a few alternatives for QPU control and data acquisition, starting with QCoDeS,[54] which is a Python framework developed by Microsoft Quantum. Another example of control and data acquisition software is ARTIQ,[7] which is also used to control hardware, as discussed in Sec. II B 4. Control and data acquisition software encompasses various elements and instruments: the quantum device, the firmware that interconnects with the instrument drivers, and parameters that are controlled and measured to generate datasets, which are stored, analyzed, and explored via data visualization tools, as depicted in Fig. 3.

To achieve frictionless control of a QPU, the software typically abstracts away the hardware elements using drivers. For example, in the QCoDeS library,[54] an instrument driver is a Python class that implements methods to operate the hardware using commands that are sent to the device via the programming interface. Namely, a quantum hardware instrument, such as an arbitrary waveform generator, can be used to generate a radio frequency pulse for the rotation of a qubit, e.g., an X gate. In order to program this pulse, the driver implements parameters such as frequency, amplitude, phase, and pulse time. The driver will then use these values to construct the right programming commands to send to the instrument to set up the pulse. The QCoDeS documentation[54] contains driver examples to control over 50 instruments.

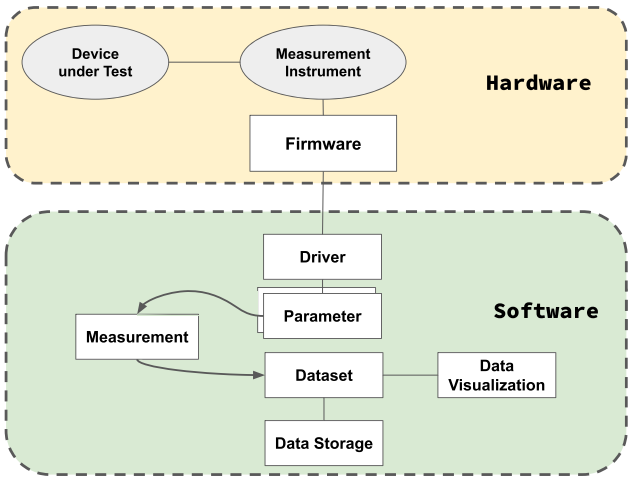


FIG. 3. Control and data acquisition. The diagram shows some of the most important elements in the software and hardware stack involved in a quantum experiment. The hardware part of the stack includes the device under test or operation, the measurement instrument, and the firmware. A driver provides interaction between the software and hardware. Drivers implement parameters that can be controlled (set) or read (get) over several measurements. The result of each measurement is stored in a dataset that is read for data analysis, visualization, and storage.

The data acquisition software uses instrument drivers to form an abstraction layer for the several pieces of hardware that control a QPU. Parameters can also be used at higher levels of abstraction on top of instrument parameters to represent QPU device elements that are controlled and read via the hardware layer, such as a qubit readout circuit. The parameters are then used in automated measurement routines with the goal to characterize, calibrate, or program a QPU or its different elements. This requires saving, analyzing, and visualizing the data that are recorded by these instruments. Most data acquisition software frameworks, therefore, also include data storage abstractions and data visualization tools. In order to control the QPU elements using quantum programming instructions, the control signals need to be calibrated for optimal gate and read-out fidelity. As shown in Fig. 1, calibration values are obtained via device testing and calibration measurements using the data acquisition software, analysis routines, and data storage in the cloud. More details can be found in Sec. II B 4 on optimal control, calibration, and characterization for high-fidelity quantum operations.

A way to approach the software–hardware interface is through an Instruction Set Architecture (ISA), as it is performed in classical computing. The ISA acts as a contract between hardware and software by explicitly describing the set of available operations, machine instruction format, supported data types, registers, and memory addressing. A well-designed ISA can present a very compact and efficient method to access the specialized features of a computing device. Graphical Processor Units (GPUs)[96] and their success in becoming an integral part of many modern devices is an example of instruction set design and standardization leading to widespread specialized architecture adoption.

There are a few quantum ISA implementations,[97–99] some of which explore this control and data acquisition approach, such as QUASAR,[100] developed at the Advanced Quantum Testbed (AQT) at Lawrence Berkeley National Laboratory (LBNL). QUASAR is based on the RISC-V ISA[101]—an open-source architecture that revolutionized the classical computing field and continues to expand toward emerging technology applications. QUASAR has been demonstrated in conjunction with QubiC[85] (QubiC is discussed separately in Sec. II B 2, below), executing experiments, including mid-circuit measurement and feed-forward, on superconducting quantum processors (QPU) at AQT, LBNL.

Example 2, Control and Data Acquisition: **"QUASAR: A Quantum Instruction Set Architecture."** The QUASAR development started in 2017 and went through several iterations. It is a project defining a quantum Instruction Set Architecture (ISA) to interface software with hardware. The first version was a tightly coupled extension that required significant changes to the processor microarchitecture. Later, the implementation moved towards a more decoupled approach. The current implementation is a RISC-V Rocket Core[102] extended with the QUASAR co-processor [note that RISC-V is a very popular open standard ISA in classical open hardware; RISC-V is based on the established reduced instruction set computer (RISC) principles, and the Rocket Core is a processor that implements RISC-V]. In contrast to classical instruction

04 September 2026 20:03:04

© Author(s) 2024

sets, quantum ISAs operate on different types of computations. Traditional general-purpose operations are supplemented with a set of basic quantum gates applied to direct-addressed qubits and/or qubit registers. The RISC-V core executes the main program; when a quantum instruction occurs at the fetch pipeline stage, it forwards it via the RoCC interface[103] to be decoded and executed by the QUASAR co-processor. Such an architectural solution allows the main core to continue performing computations while the quantum backend is generating the control pulses. Moreover, the RISC-V core can perform complex computations, such as floating-point arithmetic for phase estimation, and make conditional branching during algorithm execution based on the qubit measurement data. That makes the QUASAR implementation flexible enough to accommodate complex hybrid experiments for classical-quantum computations. The RISC-V Rocket core runs a Linux kernel that facilitates remote communication with the user host machine and provides additional functionalities for the quantum software stack.

2. Pulse-level control with hardware integration

In recent years, several pulse-level control software packages have been developed with the idea of providing end-users with a tool to program all the relevant device-specific physical parameters of the system.[7,39,55,79,81,104] This approach allows for a finer level of control over pulses during the application of gates and also makes it possible to directly use the Hamiltonian of the system as a resource for computation. Within the gate-level framework of quantum circuits, pulse-level control and simulation enables a greater level of flexibility and the ability to implement optimal control schemes. Within the analog quantum simulation framework, pulse-level control notably allows practitioners to take advantage of the mimetic capabilities of the hardware. Exposing the hardware controls at such a low level is intended to help quantum developers design software procedures while keeping the specific characteristics of the hardware in mind.[105]

Control hardware, e.g., field programmable gate arrays (FPGAs), arbitrary waveform generators (AWGs), and sequencers, have historically been closed-source, either provided by industrial manufacturers or by in-house developed systems. The ARTIQ/Sinara project is a notable exception, focused on the fast control of ion-based quantum processors through FPGAs, as detailed in the box below (*Example 3*).

> *Example 3, Control and Data Acquisition:* **"ARTIQ: Advanced Real-Time Infrastructure for Quantum physics."** The ARTIQ experiment control and data acquisition system was initiated in 2013 at the NIST Ion Storage Group, in partnership with M-Labs Ltd., to address the deficiencies observed with control systems developed in-house by physicists or based on existing commercial solutions. A key feature of the ARTIQ system is a high-level Python-based programming language that helps describe complex experiments, which is compiled and executed on dedicated FPGA hardware with nanosecond timing resolution and sub-microsecond latency. Using the abstractions provided by Python, ARTIQ has the capability to handle the entire control stack, from quantum circuits to pulse-level programming. ARTIQ also supports connecting several FPGAs together and synchronizing their clocks in the sub-nanosecond regime, which greatly expands the input/output (I/O) scalability of the system.
>
> Initial versions of the ARTIQ system ran on physicist-designed hardware based on FPGA development kits with custom I/O expansion boards. In order to improve the quality, availability, and reproducibility of the hardware, the Sinara project[60] was started in collaboration with Warsaw University of Technology. The Sinara project developed a modular system with carrier FPGA cards (Kasli, Kasli-SoC) controlling various so-called Eurocard Extension Modules (EEMs) catering to the needs of each experiment—such as digital I/O, Analog to Digital Converter (ADC), Digital to Analog Converter (DAC), Direct Digital Synthesis (DDS), Phase-Locked Loop (PLL) synthesizer, and AWG. There is ongoing work by Duke University on firmware[52] and by Warsaw University of Technology on hardware to port ARTIQ to the RF-SoC platform, with similar capabilities to QICK.
>
> Combined with the ARTIQ software, the Sinara hardware has encountered substantial success, with almost a 1000 quantum physics experiments relying on ARTIQ/Sinara systems.
>
> ARTIQ, including its gateware, the firmware, and the ARTIQ tools and libraries, are licensed as LGPLv3+. The Sinara hardware designs are licensed under the CERN OHL. This ensures that a user of ARTIQ or Sinara hardware designs obtains broad rights to use, redistribute, study, and modify them.

More recently, other FPGA-based projects have emerged beyond ARTIQ for SC qubits control, such as QubiC[57] and QICK,[55] two projects based on software, firmware, and hardware composed of radio-frequency system-on-chip (RF-SoC) boards. Open-source FPGA-based control efforts for SC qubits started in 2018 with QubiC, developed at the Advanced Quantum Testbed (AQT) based at LBNL (more information on this facility is given in Sec. II D 2, discussing testbeds). The first version of QubiC was implemented on the Xilinx VC707 platform,[85] informed by actual quantum information science experiments at AQT. To reduce cost, complexity, and space requirements, customized hardware components were designed and fabricated, such as in-phase and quadrature (I/Q) mixing modules integrating filters, amplifiers, and bias_Ts on printed circuit boards (PCBs) with Electromagnetic Interference (EMI) shielding, the design of which was published and open-sourced to benefit the community.[106] Additionally, automated single- and two-qubit gate calibration protocols were developed using QubiC.[57]

To take advantage of the growing capabilities of recent RF-SoC platforms, QubiC was then ported to the Xilinx ZCU216 platform, capable of direct generation of control pulses at higher

04 September 2026 20:03:04

© Author(s) 2024



FIG. 4. QubiC, through the years: QubiC 1.0, initially implemented on the Xilinx VC707, in its chassis with auxiliary components (left), the Analog Front End chassis for QubiC 1.0 (middle), and QubiC 2.0,[108] implemented on the Xilinx ZCU216 with a custom SMA fanout board. A customized Surface Mount Assembly (SMA) fan-out board has been designed and recently fabricated to fully utilize all the channels of the ZCU216 board. Separately, a low-cost DAC extension was developed for the VC707 platform to meet the varying frequency needs of different superconducting QPU architectures. The QubiC team has recently demonstrated heterogeneous synchronization of two sets of VC707 boards with low-cost DACs and two ZCU216 platforms.

frequencies (up to 10 GHz in the second Nyquist zone). Additionally, a novel distributed processor design was created and implemented to enable distributed decision-making and branching, such as mid-circuit measurement and feed-forward.[107] The different iterations of the QubiC 1.0 and QubiC 2.0[108] implementations are shown in Fig. 4. To form a prototype full-stack quantum control system with increased potential for future experiments, QubiC and QUASAR (described earlier in Sec. II B 1) have been integrated and applied to quantum computing experiments at AQT and LBNL.[109]

Boards from the QICK project have reached over 40 labs in the USA and abroad after a little more than two years. Using QICK helps reduce equipment costs and increases the performance of quantum information science experiments with SC qubits.[110] All of the QICK implementations have been on Zynq boards, with the firmware provided by PYNQ (Python for Zynq) compatible with multiple generations of RF-SoC FPGAs. An image of a QICK board is shown in Fig. 5. The usage of QICK has already expanded from SC qubits to atomic, molecular, and optical physics qubits and spin qubits (Nitrogen-Vacancy-center qubits) on the hardware side. On the application side, QICK is used not only for quantum computing but also for quantum sensing experiments, e.g., for dark matter candidate detection,[111] as well as for RF-SoC control in particle physics detection beyond quantum technology.

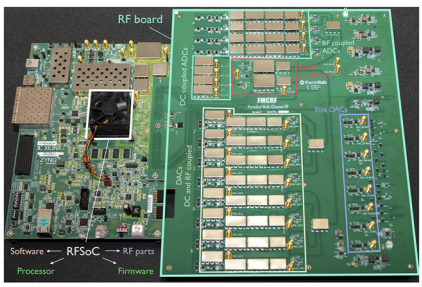


FIG. 5. The Quantum Instrumentation Control Kit (QICK). The QICK consists of two pieces of hardware: a commercial RFSoC evaluation board (left), which connects to the QICK RF&DC custom board (right), which can be used for additional signal amplification and filtering. QICK supports the AMD-Xilinx ZCU111[112] (shown in the picture with its companion custom board) and the AMD-Xilinx ZCU216[113] (not shown) with a new companion board under fabrication and testing.

3. Pulse-level simulation

Simulation is important to design an experiment before running it on hardware and then to validate and interpret the results after data collection. Pulse-level simulation can be employed for quantum optics experiments, quantum simulation, and both gate-level (digital) quantum computing and analog quantum computing. In quantum computing, although gate-level instructions are the most common formalism on the user-side to write quantum algorithms, there can be a compiler that transforms discrete operations into pulses. Examples of pulse-level simulators include Pulser[81] and Bloqade.jl, two toolboxes for neutral-atom QPUs, respectively, developed in Python and Julia. qutip-qip is a simulation toolkit that provides the freedom to design any QPU and simulate pulse-level execution, integrating QuTiP's noisy time evolution.[39] We summarize information on the Pulser in *Example 4*.

> *Example 4, Pulse Level Simulation:* **"Pulser: A library for pulse-level/analog control of neutral atom devices."** The Pulser framework is an open-source software library for designing pulse sequences for neutral-atom QPUs.[50] In such devices, individual atoms are placed in arrays of microscopic traps,[114–116] and their quantum state is manipulated through the application of laser or microwave pulses. Using Pulser, developers can control all the relevant physical parameters of the qubit register and the driving channels.
>
> The central object in Pulser is called a sequence, which is linked to a device. The device contains information about the hardware constraints, such as the maximal number of qubits available, the minimal pairwise distance between qubits that can be reached, or the number of lasers and the maximal amount of laser power available for each of them. These constraints are enforced upon the register, where the neutral-atom array is defined, and upon the pulses that are added to the sequence. Each pulse is defined by its phase, amplitude, and detuning waveforms,

04 September 2026 20:03:04

© Author(s) 2024

and pulses are sequentially added to the channels that are available on the device. The resulting program can subsequently be sent to QPUs and executed on the hardware after a device-specific calibration-aware compilation step. In addition to providing an interface to hardware, Pulser includes a built-in emulator relying on QuTiP[34] that faithfully reproduces the hardware behavior. To this extent, Pulser also acts as a simulator for QPU design.

4. Optimal control, calibration, and characterization

High fidelity operations require high fidelity optimized controls. The process of generating optimal pulses traditionally involves a model-based open-loop optimal control step (in simulation) such as GRAPE, Krotov, or GOAT algorithms,[117–119] followed by a model-free closed-loop calibration step (in hardware). The former relies on building a sophisticated model of the system, and the optimality of the generated pulses is constrained by the model's capability to faithfully reproduce the imperfections of open quantum systems. The latter usually involves the use of a gradient-free optimization algorithm, e.g., Nelder–Mead or CMA-ES, to optimize the parameters of an open-loop optimal pulse on the quantum hardware benchmarked with a fidelity measure such as ORBIT.[120] Open-source libraries such as JuliaQuantumControl, QuOCS, and C3-Toolset[82,83] aim to provide a broad range of the optimal control functionalities previously discussed with easy-to-use interfaces in Python or Julia. These tools also incorporate automatic differentiation (AD) techniques, which have made it possible to obtain gradients of arbitrary order for free, even with the most complex numerical simulations. Such gradients are essential for the optimization of pulse parameters in open-loop control. AD capabilities are provided by the use of standard machine learning frameworks such as TensorFlow or JAX.[121]

Example 5, Control and Data Acquisition: **"Qudi: A modular Python suite for experiment control and data processing."** Qudi provides a flexible and modular framework for controlling and analyzing data from confocal microscopy, quantum optics, and quantum information experiments, primarily based on color centers in diamond. Developed using Python and Qt, it is a feature-rich open-source solution providing a GUI, real-time data visualization, distributed execution over networks, a Jupyter Notebook interface, configuration management, and data recording.[58] A key feature of Qudi is its modular design, consisting of functionally independent parts that are orchestrated by a manager component with a declarative configuration management system. This also makes it very easy to extend the framework to support new devices and experiments. The entry point for any user is the start.py file, which loads the core components that take care of configuration parsing, module management, error logging, and remote network access. There is a three-tier architecture consisting of logic modules, hardware modules, and GUI modules, ensuring a clear separation of concern and allowing independent use, development, and deployment.

In order to improve the fidelity of the pulse obtained through open-loop control, both the model parameters and the model itself need refinement. One approach for refining model parameters is the technique of data-driven system identification, also known as characterization. This is achieved through learning model parameters from data collected during experiments performed on the hardware, e.g., the calibration step previously outlined. Refining the model translates to building a complex digital twin that models not only the quantum dynamics but also all of the accessory classical electronics and their non-ideal behavior. These two techniques—model learning and quantum-classical digital twin—are tightly integrated as a unified solution in the C3-Toolset package,[82] as discussed below (*Example 5*).

Example 6. Optimal Control, Calibration, and Characterization: **"C3: An integrated tool-set for control, calibration, and characterization."** The C3-Toolset package provides an API for directly interfacing with hardware to use the pulses generated by optimal control and further optimize them using closed-loop calibration algorithms. Users have access to a high-level Qiskit interface that allows them to run gate or pulse level quantum circuits on the full-physics noisy high-fidelity differentiable simulator. Libraries like QuOCS[122] also provide an extensive suite of model-based optimal control and pulse-shaping for a wide variety of hardware platforms. In recent years, a variety of machine learning techniques for quantum control have been closely integrated into the quantum device bring-up process. Besides the previously mentioned process of learning model parameters, reinforcement learning (RL) has been particularly useful in a variety of applications. The open-source library rbqoc[123] details a technique for noise-robust control using RL.

Calibration is generally needed for most platforms. Depending on the architecture, this involves different control signals. A difference between academic research labs and industry is the tendency to automate and standardize calibration tasks. Tests are run periodically to validate the status of the system before and during an experiment. On a schematic level, these can be seen as continuous integration (CI) tests involving hardware control but similar to open-source testing for software projects. If the job does not pass a test, automatic re-calibration is prompted.

C. Designs for quantum error correction

Many applications of quantum computing, networking, and sensing technologies will require a significant reduction in error rates. In the long term, developing quantum hardware with built-in error correction can address this need through fault-tolerant quantum computing. In this section, we review some of the open-source software available at present for studying and simulating quantum error correcting codes. This software is becoming increasingly important as quantum error correction moves from theory into practice.

04 September 2026 20:03:04

© Author(s) 2024

Quantum error correction codes are often based on stabilizer subsets of quantum computation and can benefit from specialized simulators. A leading open-source package for stabilizer simulation at present is Stim,[124] which improves on existing stabilizer simulators available in Qiskit, Cirq, or other packages.

Other libraries sit on top of simulators like Stim and allow users to explore quantum error correcting code designs. These packages help inform longer term planning for quantum processor architecture. Examples include plaquette[125] and stac.[126] Both of these are Python libraries that have built-in examples of quantum error correcting codes and allow you to generate logical circuits to study performance. There are also tools that act as compilers, translating logical circuits into physical circuits that can be run directly on capable hardware, e.g., the suite of tools developed at latticesurgery.com for compiling to the surface code.

Finally, there are open implementations of the decoding algorithms used to decide how to best correct errors detected by different quantum codes. PyMatching[127,128] is an example of an open-source decoder. This software can help inform the design of processors specifically adapted to run quantum error correction. For example, stim was used to simulate novel superconducting qubit designs to support surface code parity measurements in Ref. 129.

There remain many important opportunities to improve quantum error correction. For example, fast classical control of QPUs is essential for feedback loops in quantum error correction implementation. Tailored tools could reduce the existing gap between hardware operations and software instructions. More generally, increasing the ecosystem of open-source software available in this field will both support new breakthrough ideas as well as accelerate the transition from theory to practice in building fault-tolerant quantum computers.

D. Open quantum hardware facilities

In this section, we review the state of the art with respect to facilities that can enable a robust OQH ecosystem. We find three qualitatively different categories: open-access to QPUs and remotely accessible research labs, collaborative testbeds, and facilities for fabrication such as foundries.

1. Remotely accessible labs and cloud-connected labs

The idea of open remote labs, i.e., laboratories that can be remotely accessed by users to perform real experiments, is not new in the context of scientific research and education.[130,131] For example, the educational tool HYPATIA[132] can be used to perform particle-physics experiments with real data produced by the ATLAS experiment at CERN. In a similar way, quantum computers can be used as quantum remote labs, i.e., advanced laboratories in which quantum experiments can be performed for purely scientific purposes without any computational motivation.[63,133–139] For example, in Ref. 140, a quantum computer was used for testing quantum fluctuation theorems, while in Ref. 141 a quantum computer was used to prepare a many-body quantum system in a time-crystalline phase.

Superconducting-circuit quantum computers originally became available online from IBM Quantum with the IBM Quantum Experience, Rigetti Computing with the Rigetti Quantum Cloud Services, and other providers. These have had an impact on the way research is performed in quantum computing and quantum optics as well as enabling access for students for outreach activities, due to the fact that experiments can be performed much more easily (even by theorists), from the cloud. We are now in a second phase with more providers putting their devices online, encompassing more technologies—ion-based (IonQ, Quantinuum), atom-based (e.g., Infleqtion, PASQAL, and QuEra), photonics-based (e.g., Quandela and Xanadu), quantum-dot based, etc.

Turning attention to institutional frameworks, the European OpenSuperQ project (and the follow-on OpenSuperQPlus project)[61] is similarly aimed at developing public-owned full-stack open-access superconducting systems. Multiple remotely accessible QPUs are online or currently being set up as part of this project at the Delft University of Technology, the Wallenberg Center for Quantum Technology (Chalmers), the Walther-Meissner-Institut (Munich), and at the Forschungszentrum Juelich (from phase 1). A hybrid form of cloud access is also emerging, in which cloud providers give access to different quantum processor providers. Furthermore, sustaining open access to QPU providers can be important for scientific discovery.

Over the years, the level of control over device properties in compilation, optimization, qubit mapping, etc. has increased, spilling notably into remote pulse level control, both for digital (e.g., Qiskit pulse[79]) and analog quantum computing (e.g., PASQAL's Pulse studio[115]). The deeper the access, the more advanced the control researchers have over hardware and the greater the opportunity for integration of open-hardware features with cloud access. An example of a novel interaction between cloud providers and OQH is given by the Amazon AWS application developed for QICK, called SideQICK, which is being integrated into a more general cloud queue for quantum devices.[142]

We also witness examples of a hybrid model of industry-public interaction enabling open access. Quantum computers are being shipped to research centers and further integrated with HPC centers, which also act as simulator infrastructure (such as the EU HPCQS). This provides new avenues for industry-academic collaborations and purpose-specific hardware customization, use cases, and access. It will be important to foster academic research labs putting their quantum devices online, building upon the open-source tools described in Secs. II A–II D. This could further change the research landscape, enabling more researchers to test research ideas on more devices and novel architectures and platforms, which in turn would facilitate technology transfer.

2. Deeply collaboratively testbeds

While cloud-based platforms allow scientists and the public to submit their circuits and receive the results, deeper, customized access to the full stack to enable more involved experiments and R&D efforts is generally not possible on these platforms. To provide such access to users in academia, industry, and national labs, the US Department of Energy has funded two testbed programs: QSCOUT (Quantum Scientific Computing Open User Testbed), based on trapped ions and located at Sandia National Laboratories, and AQT (Advanced Quantum Testbed), based on superconducting qubits and located at Lawrence Berkeley National Laboratory. A similar network called Qu-Test,[143] managed by TNO at Delft,

04 September 2026 20:03:04

© Author(s) 2024

Netherlands, has recently been kicked off with the goal of providing a federated network of testbeds by bringing together 13 service providers and 11 industrial users from the European quantum community. Likewise, in the UK, the National Quantum Computing Center (NQCC) is building, testing, and hosting full-stack quantum computing solutions.[64] Through close collaboration between industry, government, and academia, the NQCC is initially focusing on developing demonstrator platforms that provide direct access to quantum computing resources while also helping to develop a robust supply chain. A recently concluded funding call by Innovate UK[144] earmarked 30×10^6 GBP to build, commission, and validate an operational quantum computing testbed in the UK as part of the NQCC.

In Canada, the Open Quantum Design (OQD) initiative aims at building a completely open-source and open-hardware quantum computer based on ions. OQD has taken the form of a full-fledged non-profit organization arising from previous experimental and theoretical work performed at the Institute for Quantum Computing and Perimeter Institute.

These platforms allow low-level access to the full stack of quantum hardware (including programming languages,[145] gate-pulse shaping,[104] noise injection, unique qubit/qudit states, specialized calibrations, comparison of compilation techniques, etc.), allowing users to probe how the machines actually work and how to make them work better. The testbeds foster deep collaboration between the host laboratory and the users.

Collaborations on these testbeds have so far resulted in demonstrations of scientific applications of near-term quantum computers and the development of many tools to benchmark quantum computers as well as to characterize and mitigate errors on their QPUs. This could further change the research landscape, enabling more researchers to test research ideas on more devices. Moreover, the existence of testbeds would be crucial to enable reproducible benchmarks for quantum hardware and for application-driven tasks. Testbeds are key to facilitate startup creation in the quantum space, as they lower the barrier for testing prototypes with expensive equipment (such as dilution fridges).

3. Fabrication and foundries

As described in Sec. II A, open-source tools exist to facilitate the design and creation of quantum processors *in silico*. However, translating those designs into actual hardware can be difficult and expensive. Current arrangements for producing hardware on the basis of designs fall into four categories: partnerships, the construction of research-grade fab facilities, the use of academic foundries, and "quantum-fab-as-a-service."

Several companies in recent years have begun partnerships with major manufacturers to procure supply lines for their hardware designs. For example, photonic startups PsiQuantum and Xanadu have both signed agreements with semiconductor manufacturers GlobalFoundries.[146,147] Trapped-ion startup IonQ has sourced some of its traps from Sandia National Laboratories.[63] These sorts of partnerships are typically only available to startups or other corporations and allow for the use of existing manufacturing techniques and scalable processes. However, whether a rapid prototype of hardware can be achieved is unclear.

An alternative approach is to set up one's own fabrication facility. Startup Rigetti Computing is notable in this space for their "Fab-1" facility,[148] which allows the company to prototype new hardware. However, such a facility can be expensive to construct and requires access to large amounts of capital. In the United States, in recognition of the need for fabrication facilities, various foundries are being set up around the country, including the UCSB Quantum Foundry,[149] MonArk Quantum Foundry,[150] and the LPS Qubit Collaboratory.[151] The European Commission has established a somewhat similar program called Qu-Pilot[143] under the umbrella of the Quantum Flagship, consisting of 21 partners from nine different countries, with the goal of developing and providing access to federated European production facilities that link existing infrastructure. The overall coordination of the project is managed by VTT, Finland.

Finally, some startups have leveraged this need toward the development of new businesses. For example, startup QuantWare in the Netherlands[152] helps designing, developing, and fabricating hardware for customers. In Canada, the NSERC CREATE programs have partnered with CMC Microsystems for novel superconducting circuit workshops at the Stewart Blusson Quantum Matter Institute. In the future, more industry players, industry consortia, startup incubators, and academic partnerships could enable even more facilities for processor fabrication, inspired by existing electronics industry frameworks such as the Efabless and the Google Silicon project.

III. DISCUSSION: CURRENT GAPS AND FUTURE RECOMMENDATIONS

In Sec. II, we reviewed OQH at present, with an overview of the various OQH categories and deep dives into selected projects. From this overview, it is possible to draw an overarching picture and outline some major topics across the OQH ecosystem. In Secs. III A–III H, we list these topics, identifying gaps and making recommendations to close them.

A. OQH growth and maintenance across technologies and architectures

Currently, most of the OQH projects focus on tools for quantum computers. There are considerable opportunities for expanding OQH projects and tools to accelerate scientific discovery and tech transfer in quantum communication, quantum metrology, and quantum sensing. Moreover, given the early stage of the field, for specific functionalities, tooling supporting only a given set of architectures may be currently available. For example, the open hardware tools for chip design resolve mostly around SC qubits (and partially ion traps). There is room for tools for other QPU architectures, such as atom-based, spin-based, and photonics-based QPUs. With respect to control, open hardware projects, including firmware and hardware such as FPGAs, have been developed mostly for ion traps and SC qubits and can be more broadly applied to other architectures. Moreover, while it is possible to find multiple open-source FPGAs (and general open-hardware electronics projects, many of which are applicable to physics), one would be hard-pressed to find anything that comes closest to an optical frequency comb in photonics tooling.

B. APIs and standards for instruction sets

We note that interoperability is a major challenge in the OQH ecosystem. From APIs in the software stack to software–hardware

04 September 2026 20:03:04

© Author(s) 2024

integration via ISA, there is work to be performed. Currently, there is considerable duplication on the higher software stack, and one needs to do conversions of quantum programs in order to run them on different QPUs. Frameworks such as the Quantum Intermediate Representation (QIR) project and OpenQASM (Open Quantum Assembly Language)[153,154] can help at a higher level. Broader support for common frameworks getting further down at the hardware control level would be desired, such as QUASAR and QUA.

C. Benchmarks

Open-source benchmarking suites can have a strong impact on the scientific community, providing information on the state of the art of current hardware architectures. So far, we have noted that quantum technologies have been confronted with a lack of standardized benchmarks. Benchmarks are useful for evaluating the performance of quantum devices and algorithms, as well as assessing their relative performance compared to purely classical solutions in relevant applications or non-conventional computers.[155] Thankfully, efforts to standardize benchmarks, such as those provided by the QED-C, a U.S.-based industry consortium,[42–44] have been occurring. Pipelines for crowd-sourced benchmark results are also needed: https://metriq.info/ is one attempt, inspired by projects such as Papers With Code and MLCommons in machine learning. Pipelines, including OSS and OQH, can simplify the accomplishment of standardized benchmarks.

D. Open access to hardware

Currently, there are several cloud-accessible QPUs and cloud service frameworks. Providers have made QPUs accessible in a variety of ways, including direct and free access, as well as through partnerships within institutional frameworks [e.g., ORNL's Quantum Computing User Program (QCUP) for national labs and federally funded consortia]. However, the research community would benefit from further access enabled by grants and other awards. Ensuring such frameworks and access points increase as soon as possible is necessary for creating a more equitable research landscape. Moreover, while several research labs have infrastructure to enable remote access and data acquisition for experiments, this is generally for internal use. Sometimes, access is extended to partnering organizations or collaborating researchers. However, there is a general lack of research labs that have put their devices online for cloud access for a wider community of users and researchers. This is due to the overhead in infrastructure involved in making such devices operational and maintained. OQH can help with this regard by providing researchers with existing tools that can be plugged in to bring up experiments from research labs.

E. Reproducibility

In order to facilitate the reproducibility of results, journals and academic guidelines can further encourage (and mandate, where appropriate) the sharing of open software along with research publications. Funding agencies, universities, and other research stakeholders can facilitate these efforts by recognizing software artifacts as important, citable outcomes for measuring research impact.

F. OQH policy and intellectual property

Implementing a policy that is open-hardware aware can favor the growth of an OQH ecosystem. For example, learning from existing open hardware projects beyond quantum technology can help avoid known pitfalls. A practical example is the adoption of an integrated toolchain that fosters collaboration between academic labs, software developers, and industry. An example at the facility level is the investment in the establishment of shared facilities, such as quantum foundries, which can provide a flywheel effect for startups. Policymakers and quantum experts have the opportunity to work together in order to ensure that informed decisions are made about the application of export control and intellectual property protection laws to quantum hardware. On the one hand, this could ensure that policymakers provide clear guidance about how efforts to open up quantum hardware will be treated from the perspective of these laws. On the other hand, researchers can develop expertise on the correct use of licenses for open-source software and open hardware. This guidance can include implementing processes through which one can more easily assess what hardware-based technology should be open-sourced and what should not, making strategic calls at a narrow level (researcher, company) and a wider level (common good for society). Researchers need to balance intellectual property (IP) and strategic competitive considerations for certain components and technologies that could provide commercial exploitation. At the same time, they can consider when OSS can be used in business models, helping grow a community of users and creators around products.

G. Support of OQH in education and academia

We believe it is important to foster OSS and OQH adoption in education and research. One actionable item is to encourage schools and universities to include in course material and labs the usage of open-source software[156] and open hardware.[157,158] Programs that prepare the future quantum workforce are fundamental to broaden the acquisition of talents and develop new career paths.[5,6,24] This includes developing communities across domain knowledge.

A bottom-up approach to strengthening OQH is by encouraging academic labs to pull together their systems with existing open-source tools so they can create do-it-yourself small-scale testbeds. If open-source tools are not available, researchers can consider developing and open-sourcing new tools or using less-closed tools (e.g., proprietary but with a layer of open APIs). Fostering the creation of new open-hardware projects includes engaging in activities, programs, and material that facilitate the open-sourcing of existing projects, such as Ref. 159, which provides information about starting a scientific OSS project. Starting an OQH project includes general challenges found in other OSS projects (testing, documentation, distribution, maintenance, and community relations) and some specific ones, such as how to distribute physical devices to users or work with experimental labs on ensuring device control.

An example of a top-down approach to supporting open-source projects (and open hardware) is provided by funding agency programs acknowledging the impact of OSS projects in science. One notable example is the NSF POSE program. Furthermore, tailored funding in programs focused on OQH could strengthen the ecosystem by addressing some of its characteristics.

04 September 2026 20:03:04

© Author(s) 2024

H. Community building: projects support and governance

Similarly to other OSS projects, OQH projects also need to interact with their community of users, engage feedback, and create guidelines to foster code contributions and meetings. They can benefit from non-profit organization support for their governance, as typically scientific OSS projects start in a research lab or group and then grow in scope and require the representation of more parties and partners. There is a need to simplify the bureaucracy in compliance at the institutional and government level to contribute to open-source projects and open hardware, e.g., by equipping national labs with default policies that would allow researchers to contribute to open-source projects with permissive licenses approved by the Open Source Initiative (OSI). Often, the originating organization such as a national lab has restrictions that prevent an open-source project from growing independently. This highlights the need for alternatively supporting organizations to incubate and house open-source projects. Examples of non-profit organizations that do this in the classical space are NumFOCUS, the Linux Foundation, and the Open Source Hardware Association (OSHA). The Unitary Fund is a 501(c)(3) non-profit organization that offers this support specifically to quantum technology projects.

IV. CONCLUSIONS

Quantum computers—and quantum devices in general—provide a fascinating prospect to bridge a platform for fundamental science, technology transfer in engineering, and novelties in cloud computing and actuators. An open hardware approach in quantum technology will preserve, and possibly expand, the pre-competitive space necessary to make ideas flourish and enable accessible benchmarks and evaluations that have already characterized quantum software.

As we have illustrated, quantum technology provides unique challenges and features with regard to the usability and feasibility of the open-hardware stack: from process design toolkits to foundries, from simulation software to control and data acquisition, from cloud infrastructure to testbeds. All these layers can benefit from collaborative innovation. Learning from lessons learned in the early days of quantum software and from other open hardware verticals in conventional computing and tooling, as well as from a snapshot of current gaps in the quantum ecosystem, we hope that the quantum industry can efficiently implement innovation, build lively communities, and expand its workforce.

ACKNOWLEDGMENTS

This work stemmed from a workshop[10] hosted at IEEE QCE 2021. The authors acknowledge Gary Steele and Zlatko Minev for fruitful discussions. N.S. acknowledges the organizers of IEEE QCE 2021 for the opportunity to start the discussion of OQH, which culminated in this manuscript, and many of the co-authors for their contributions at that workshop. This material was funded in part by the U.S. Department of Energy, Office of Science, and the Office of Advanced Scientific Computing Research Quantum Testbed Program. Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under Contract No. DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government. Grant No. SAND2023-09347O.

This work was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Accelerated Research in Quantum Computing under Award No. DE-SC0020266. A.M. acknowledges support from the PNRR MUR Project No. PE0000023-NQSTI. QubiC and QUASAR research was funded by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research Quantum Testbed Program under Contract No. DE-AC02-05CH11231.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Nathan Shammah: Conceptualization (lead); Investigation (lead); Supervision (lead); Writing – original draft (lead). **Anurag Saha Roy**: Investigation (equal); Writing – original draft (equal). **Carmen G. Almudever**: Investigation (equal); Writing – original draft (equal). **Sébastien Bourdeauducq**: Investigation (equal); Writing – original draft (equal). **Anastasiia Butko**: Investigation (equal); Writing – original draft (equal). **Gustavo Cancelo**: Investigation (equal); Writing – original draft (equal). **Susan M. Clark**: Investigation (equal); Writing – original draft (equal). **Johannes Heinsoo**: Investigation (equal); Writing – original draft (equal). **Loïc Henriet**: Investigation (equal); Writing – original draft (equal). **Gang Huang**: Investigation (equal); Writing – original draft (equal). **Christophe Jurczak**: Investigation (equal); Writing – original draft (equal). **Janne Kotilahti**: Investigation (equal); Writing – original draft (equal). **Alessandro Landra**: Investigation (equal); Writing – original draft (equal). **Ryan LaRose**: Investigation (equal); Writing – original draft (equal). **Andrea Mari**: Investigation (equal); Writing – original draft (equal). **Kasra Nowrouzi**: Investigation (equal); Writing – original draft (equal). **Caspar Ockeloen-Korppi**: Investigation (equal); Writing – original draft (equal). **Guen Prawiroatmodjo**: Investigation (equal); Writing – original draft (equal). **Irfan Siddiqi**: Investigation (equal); Writing – original draft (equal). **William J. Zeng**: Investigation (equal); Writing – original draft (equal).

DATA AVAILABILITY

The data that support the findings of this study are available within the article.

REFERENCES

[1] W. Zeng, B. Johnson, R. Smith, N. Rubin, M. Reagor, C. Ryan, and C. Rigetti, "First quantum computers need smart software," Nature **549**, 149 (2017).
[2] M. Fingerhuth, T. Babej, and P. Wittek, "Open source software in quantum computing," PLoS One **13**, e0208561 (2018).

04 September 2026 20:03:04

© Author(s) 2024

[3]J. Zhao, "Quantum software engineering: Landscapes and horizons," arXiv:2007.07047 (2021).
[4]M. F. J. Fox, B. M. Zwickl, and H. J. Lewandowski, "Preparing for the quantum revolution: What is the role of higher education?," Phys. Rev. Phys. Educ. Res. **16**, 020131 (2020).
[5]A. Asfaw, A. Blais, K. R. Brown, J. Candelaria, C. Cantwell, L. D. Carr, J. Combes, D. M. Debroy, J. M. Donohue, S. E. Economou *et al.*, "Building a quantum engineering undergraduate program," IEEE Trans. Educ. **65**, 220 (2022).
[6]A. S. Dzurak, J. Epps, A. Laucht, R. A. Malaney, A. Morello, H. I. Nurdin, J. J. Pla, A. Saraiva, and C. H. Yang, "Development of an undergraduate quantum engineering degree," IEEE Trans. Quantum Eng. **3**, 6500110 (2022).
[7]S. Bourdeauducq, R. Jördens, P. Zotov, J. Britton, D. Slichter, D. Leibrandt, D. Allcock, A. Hankin, F. Kermarrec, Y. Sionneau, R. Srinivas, T. R. Tan, and J. Bohnet, *Artiq 1.0*, 2016.
[8]Z. K. Minev, Z. Leghtas, S. O. Mundhada, L. Christakis, I. M. Pop, and M. H. Devoret, "Energy-participation quantization of Josephson circuits," arXiv:2010.00620 [quant-ph] (2021).
[9]T. L. Scholten and D. Greenberg, in Workshop on Software for Quantum Applications, Algorithms, and Workflows at QCE20, 2020.
[10]T. L. Scholten and N. Shammah, in Workshop on Open Quantum Hardware: Accelerating the Control, and Use, of Quantum Computing Systems at QCE21, 2021.
[11]M. Köfferlein, *KLayout*, 2007.
[12]R. Thew, T. Jennewein, and M. Sasaki, "Focus on quantum science and technology initiatives around the world," Quantum Sci. Technol. **5**, 010201 (2019).
[13]T. Roberson and A. G. White, "Charting the Australian quantum landscape," Quantum Sci. Technol. **4**, 020505 (2019).
[14]Q. Zhang, F. Xu, L. Li, N.-L. Liu, and J.-W. Pan, "Quantum information research in China," Quantum Sci. Technol. **4**, 040503 (2019).
[15]M. Riedel, M. Kovacs, P. Zoller, J. Mlynek, and T. Calarco, "Europe's quantum flagship initiative," Quantum Sci. Technol. **4**, 020501 (2019).
[16]E. Gibney, "Europe's billion-euro quantum project takes shape," Nature **545**, 16 (2017).
[17]P. Knight and I. Walmsley, "UK national quantum technology programme," Quantum Sci. Technol. **4**, 040502 (2019).
[18]Y. Yamamoto, M. Sasaki, and H. Takesue, "Quantum information science and technology in Japan," Quantum Sci. Technol. **4**, 020502 (2019).
[19]A. Fedorov, A. Akimov, J. Biamonte, A. Kavokin, F. Y. Khalili, E. Kiktenko, N. Kolachevsky, Y. Kurochkin, A. Lvovsky, A. Rubtsov *et al.*, "Quantum technologies in Russia," Quantum Sci. Technol. **4**, 040501 (2019).
[20]C. I. Merzbacher, "US quantum initiatives: From R&D to innovation," Proc. SPIE **11540**, 1154003 (2020).
[21]M. G. Raymer and C. Monroe, "The US National Quantum Initiative," Quantum Sci. Technol. **4**, 020504 (2019).
[22]C. Monroe, M. G. Raymer, and J. Taylor, "The U.S. National Quantum Initiative: From Act to action," Science **364**, 440 (2019).
[23]B. Sussman, P. Corkum, A. Blais, D. Cory, and A. Damascelli, "Quantum Canada," Quantum Sci. Technol. **4**, 020503 (2019).
[24]C. D. Aiello, D. D. Awschalom, H. Bernien, T. Brower, K. R. Brown, T. A. Brun, J. R. Caram, E. Chitambar, R. Di Felice, K. M. Edmonds, M. F. J. Fox, S. Haas, A. W. Holleitner, E. R. Hudson, J. H. Hunt, R. Joynt, S. Koziol, M. Larsen, H. J. Lewandowski, D. T. McClure, J. Palsberg, G. Passante, K. L. Pudenz, C. J. K. Richardson, J. L. Rosenberg, R. S. Ross, M. Saffman, M. Singh, D. W. Steuerman, C. Stark, J. Thijssen, A. N. Vamivakas, J. D. Whitfield, and B. M. Zwickl, "Achieving a quantum smart workforce," Quantum Sci. Technol. **6**, 030501 (2021).
[25]P. Mirowski, "The future(s) of open science," Soc. Stud. Sci. **48**, 171 (2018).
[26]A. Powell, "Democratizing production through open source knowledge: From open software to open hardware," Media, Cult. Soc. **34**, 691 (2012).
[27]R. Viseur, "From open source software to open source hardware," in *IFIP International Conference on Open Source Systems* (Springer, 2012), pp. 286–291.
[28]K. P. Gerhardt, E. J. Olson, S. M. Castillo-Hair, L. A. Hartsough, B. P. Landry, F. Ekness, R. Yokoo, E. J. Gomez, P. Ramakrishnan, J. Suh, D. F. Savage, and J. J. Tabor, "An open-hardware platform for optogenetics and photobiology," Sci. Rep. **6**, 35363 (2016).
[29]F. Morreale, G. Moro, A. Chamberlain, S. Benford, and A. P. McPherson, "Building a maker community around an open hardware platform," in *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems, CHI'17* (Association for Computing Machinery, New York, NY, USA, 2017), pp. 6948–6959.
[30]K. Blind, M. Böhm, P. Grzegorzewska, A. Katz, S. Muto, S. Pätsch, and T. Schubert, The impact of open source software and hardware on technological independence, competitiveness and innovation in the EU economy, 2021.
[31]A. Soriano, L. Marín, M. Vallés, A. Valera, and P. Albertos, "Low cost platform for automatic control education based on open hardware," IFAC Proc. Vol. **47**, 9044 (2014), 19th IFAC World Congress.
[32]P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nat. Methods **17**, 261 (2020).
[33]C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," Nature **585**, 357 (2020).
[34]J. Johansson, P. Nation, and F. Nori, "QuTiP: An open-source python framework for the dynamics of open quantum systems," Comput. Phys. Commun. **183**, 1760 (2012).
[35]J. Johansson, P. Nation, and F. Nori, "QuTiP 2: A Python framework for the dynamics of open quantum systems," Comput. Phys. Commun. **184**, 1234 (2013).
[36]N. Shammah, S. Ahmed, N. Lambert, S. De Liberato, and F. Nori, "Open quantum systems with local and collective incoherent processes: Efficient numerical simulations using permutational invariance," Phys. Rev. A **98**, 063815 (2018).
[37]N. Lambert, S. Ahmed, M. Cirio, and F. Nori, "Modelling the ultra-strongly coupled spin-boson model with unphysical modes," Nat. Commun. **10**, 3721 (2019).
[38]P. Groszkowski and J. Koch, "Scqubits: A Python package for superconducting qubits," Quantum **5**, 583 (2021).
[39]B. Li, S. Ahmed, S. Saraogi, N. Lambert, F. Nori, A. Pitchford, and N. Shammah, "Pulse-level noisy quantum circuits with QuTiP," Quantum **6**, 630 (2022).
[40]M. Banzi and M. Shiloh, *Getting Started with Arduino*, 4th ed. (Make Books—Imprint of: O'Reilly Media, Sebastopol, CA, 2022).
[41]M. Richardson and S. Wallace, *Getting Started with Raspberry PI* (O'Reilly Media, Inc., 2012).
[42]T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaise, C. H. Baldwin, K. Mayer, and T. Proctor, "Application-oriented performance benchmarks for quantum computing," arXiv:2110.03137 [quant-ph] (2023).
[43]T. Lubinski, C. Coffrin, C. McGeoch, P. Sathe, J. Apanavicius, and D. E. B. Neira, "Optimization applications as quantum performance benchmarks," arXiv:2302.02278 [quant-ph] (2023).
[44]M. Amico, H. Zhang, P. Jurcevic, L. S. Bishop, P. Nation, A. Wack, and D. C. McKay, "Defining standard strategies for quantum benchmarks," arXiv:2303.02108 [quant-ph] (2023).
[45]J. Jagatheesan, *DASQA*, 2023.
[46]J. Kunasaikaran, K. Mato, and R. Wille, "A framework for the design and realization of alternative superconducting quantum architectures," arXiv:2305.07052 [quant-ph] (2023).
[47]D. Cucurachi, A. Guthrie, J. Heinsoo, S. Inel, D. Janzsó, M. Jenei, K. Juliusson, J. Kotilahti, A. Landra, C. Ockeloen-Korppi, J. Räbinä, N. Savola, P. Smirnov, and E. Takala, *KQCircuits*, 2021.
[48]Z. K. Minev, T. G. McConkey, J. Drysdale, P. Shah, D. Wang, M. Facchini, G. Harper, J. Blair, H. Zhang, N. Lanzillo, S. Mukesh, W. Shanks, C. Warren, and J. M. Gambetta, *Qiskit Metal: An Open-Source Framework for Quantum Device Design and Analysis*, 2021.
[49]M. F. Gely and G. A. Steele, "QuCAT: Quantum circuit analyzer tool in Python," New J. Phys. **22**, 013025 (2020).
[50]H. Silvério, S. Grijalva, C. Dalyac, L. Leclerc, P. J. Karalekas, N. Shammah, M. Beji, L.-P. Henry, and L. Henriet, "Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays," arXiv:2104.15044 [quant-ph] (2021).

04 September 2026 20:03:04

© Author(s) 2024

[51] N. Killoran, J. Izaac, N. Quesada, V. Bergholm, M. Amy, and C. Weedbrook, "Strawberry fields: A software platform for photonic quantum computing," Quantum **3**, 129 (2019).
[52] A. V. Horn, *Duke Artiq repository*, 2023.
[53] L. Ella, L. Leandro, O. Wertheim, Y. Romach, R. Szmuk, Y. Knol, N. Ofek, I. Sivan, and Y. Cohen, "Quantum-classical processing and benchmarking at the pulse-level," arXiv:2303.03816 [quant-ph] (2023).
[54] J. H. Nielsen, *Qcodes/qcodes: Qcodes 0.40.0*, 2023.
[55] L. Stefanazzi, K. Treptow, N. Wilcer, C. Stoughton, S. Montella, C. Bradford, G. Cancelo, S. Saxena, H. Arnaldi, S. Sussman, A. Houck, A. Agrawal, H. Zhang, C. Ding, and D. I. Schuster, "The QICK (Quantum Instrumentation Control Kit): Readout and control for qubits and detectors," arXiv:2110.00557 (2021).
[56] K. J. Mesman, F. Battistel, E. Reehuis, D. de Jong, M. J. Tiggelman, J. Gloudemans, J. C. van Oven, and C. C. Bultink, "Q-profile: Profiling tool for quantum control stacks applied to the quantum approximate optimization algorithm," arXiv:2303.01450 [quant-ph] (2023).
[57] Y. Xu, G. Huang, J. Balewski, A. Morvan, K. Nowrouzi, D. I. Santiago, R. K. Naik, B. Mitchell, and I. Siddiqi, "Automatic qubit characterization and gate optimization with qubic," ACM Trans. Quantum Comput. **4**, 1 (2022).
[58] J. M. Binder, A. Stark, N. Tomek, J. Scheuer, F. Frank, K. D. Jahnke, C. Müller, S. Schmitt, M. H. Metsch, T. Unden, T. Gehring, A. Huck, U. L. Andersen, L. J. Rogers, and F. Jelezko, "Qudi: A modular python suite for experiment control and data processing," SoftwareX **6**, 85 (2017).
[59] S. Humpohl, L. Prediger, Pcerf, P. Eendebak, P. Bethke, A. Willmes, J. Bergmann, M. Meyer, T. Hangleiter, E. Kammerloher, qutech lab, *qupulse* (2023).
[60] P. Kulik, M. Sowiński, G. Kasprowicz, D. Allcock, C. Ballance, S. Bourdeauducq, J. Britton, M. Gąska, T. Harty, J. Jarosiński, R. Jördens, M. Kiepiela, N. Krackow, D. Nadlinger, K. Poźniak, T. Przywózki, D. Slichter, F. Świtakowski, M. Weber, A. Wojciechowski, and W. Zhang, "Latest developments in the Sinara open hardware ecosystem," in *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2022), pp. 799–802.
[61] OpenSuperQPlus, *Open superconducting quantum computers*, 2023.
[62] QuTech, *Quantum inspire* (2023).
[63] S. M. Clark, D. Lobser, M. C. Revelle, C. G. Yale, D. Bossert, A. D. Burch, M. N. Chow, C. W. Hogle, M. Ivory, J. Pehr, B. Salzbrenner, D. Stick, W. Sweatt, J. M. Wilson, E. Winrow, and P. Maunz, "Engineering the quantum scientific computing open user testbed," IEEE Trans. Quantum Eng. **2**, 3102832 (2021).
[64] NQCC, *National quantum computing centre—Our vision*, 2023.
[65] Y. Y. Gao, M. A. Rol, S. Touzard, and C. Wang, "Practical guide for building superconducting quantum devices," PRX Quantum **2**, 040202 (2021).
[66] C. Tahan, Quantum **5**, 584 (2021).
[67] X. Fu, M. A. Rol, C. C. Bultink, J. van Someren, N. Khammassi, I. Ashraf, R. F. L. Vermeulen, J. C. de Sterke, W. J. Vlothuizen, R. N. Schouten, C. G. Almudever, L. DiCarlo, and K. Bertels, "An experimental microarchitecture for a superconducting quantum processor," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-50'17* (Association for Computing Machinery, New York, NY, USA, 2017), pp. 813–825.
[68] G. Feng, S.-Y. Hou, H. Zou, W. Shi, S. Yu, Z. Sheng, X. Rao, K. Ma, C. Chen, B. Ren, G. Miao, J. Xiang, and B. Zeng, "SpinQ Triangulum: A commercial three-qubit desktop quantum computer," arXiv:2202.02983 (2022).
[69] A. Castellanos-Gomez, M. Buscema, R. Molenaar, V. Singh, L. Janssen, H. S. J. van der Zant, and G. A. Steele, "Deterministic transfer of two-dimensional materials by all-dry viscoelastic stamping," 2D Mater. **1**, 011002 (2014).
[70] J. Ziegler, T. McJunkin, E. Joseph, S. S. Kalantre, B. Harpt, D. Savage, M. Lagally, M. Eriksson, J. M. Taylor, and J. P. Zwolak, "Toward robust autotuning of noisy quantum dot devices," Phys. Rev. Appl. **17**, 024069 (2022).
[71] R. Pollice, G. dos Passos Gomes, M. Aldeghi, R. J. Hickman, M. Krenn, C. Lavigne, M. Lindner-D'Addario, A. Nigam, C. T. Ser, Z. Yao, and A. Aspuru-Guzik, "Data-driven strategies for accelerated materials design," Acc. Chem. Res. **54**, 849 (2021).
[72] W. Ma, Z. Liu, Z. A. Kudyshev, A. Boltasseva, W. Cai, and Y. Liu, "Deep learning for the design of photonic structures," Nat. Photonics **15**, 77 (2021).
[73] S. Molesky, Z. Lin, A. Y. Piggott, W. Jin, J. Vucković, and A. W. Rodriguez, "Inverse design in nanophotonics," Nat. Photonics **12**, 659 (2018).
[74] Ansys, *Ansys* (2023).
[75] S. Software, *Sonnet* (2023).
[76] CSC, *Elmer* (2023).
[77] AWS, *Palace* (2023).
[78] C. Geuzaine and J.-F. Remacle, "Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities," Int. J. Numer. Methods Eng. **79**, 1309 (2009).
[79] T. Alexander, N. Kanazawa, D. J. Egger, L. Capelluto, C. J. Wood, A. Javadi-Abhari, and D. C. McKay, "Qiskit pulse: Programming quantum computers through the cloud with pulses," Quantum Sci. Technol. **5**, 044006 (2020).
[80] H. Ball, M. J. Biercuk, A. Carvalho, J. Chen, M. Hush, L. A. De Castro, L. Li, P. J. Liebermann, H. J. Slatyer, C. Edmunds, V. Frey, C. Hempel, and A. Milne, "Software tools for quantum control: Improving quantum computer performance through noise and error suppression," Quantum Sci. Technol. **6**, 044011 (2021).
[81] H. Silvério, S. Grijalva, C. Dalyac, L. Leclerc, P. J. Karalekas, N. Shammah, M. Beji, L.-P. Henry, and L. Henriet, "Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays," Quantum **6**, 629 (2022).
[82] N. Wittler, F. Roy, K. Pack, M. Werninghaus, A. S. Roy, D. J. Egger, S. Filipp, F. K. Wilhelm, and S. Machnes, "Integrated tool set for control, calibration, and characterization of quantum devices applied to superconducting qubits," Phys. Rev. Appl. **15**, 034080 (2021).
[83] A. Saha Roy, K. Pack, N. Wittler, and S. Machnes, "Software tool-set for automated quantum system identification and device bring up," arXiv:2205.04829 (2022).
[84] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, *JAX: Composable transformations of Python + NumPy programs*, 2018.
[85] Y. Xu, G. Huang, J. Balewski, R. Naik, A. Morvan, B. Mitchell, K. Nowrouzi, D. I. Santiago, and I. Siddiqi, "QubiC: An open-source FPGA-based control and measurement system for superconducting quantum information processors," IEEE Trans. Quantum Eng. **2**, 6003811 (2021).
[86] G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, "On the co-design of quantum software and hardware," in *Proceedings of the Eight Annual ACM International Conference on Nanoscale Computing and Communication, NANOCOM'21* (Association for Computing Machinery, New York, NY, USA, 2021).
[87] L. M. Roch, F. Hã€se, C. Kreisbeck, T. Tamayo-Mendoza, L. P. E. Yunker, J. E. Hein, and A. Aspuru-Guzik, "ChemOS: An orchestration software to democratize autonomous discovery," PLoS One **15**, e0229862 (2020).
[88] M. Krenn, J. S. Kottmann, N. Tischler, and A. Aspuru-Guzik, "Conceptual understanding through efficient automated design of quantum optical experiments," Phys. Rev. X **11**, 031044 (2021).
[89] A. A. Melnikov, H. Poulsen Nautrup, M. Krenn, V. Dunjko, M. Tiersch, A. Zeilinger, and H. J. Briegel, "Active learning machine learns to create new quantum experiments," Proc. Natl. Acad. Sci. U. S. A. **115**, 1221 (2018).
[90] K. N. Smith, G. S. Ravi, T. Alexander, N. T. Bronn, A. Carvalho, A. Cervera-Lierta, F. T. Chong, J. M. Chow, M. Cubeddu, A. Hashim, L. Jiang, O. Lanes, M. J. Otten, D. I. Schuster, P. Gokhale, N. Earnest, and A. Galda, in Summary: Chicago Quantum Exchange (CQE) pulse-level Quantum Control Workshop, 2022.
[91] J. M. Perkel, "Why Jupyter is data scientists' computational notebook of choice," Nature **563**, 145 (2018).
[92] J. M. Arrazola, T. R. Bromley, J. Izaac, C. R. Myers, K. Brádler, and N. Killoran, "Machine learning method for state preparation and gate synthesis on photonic quantum computers," Quantum Sci. Technol. **4**, 024004 (2019).
[93] R. D. King, J. Rowland, S. G. Oliver, M. Young, W. Aubrey, E. Byrne, M. Liakata, M. Markham, P. Pir, L. N. Soldatova *et al.*, "The automation of science," Science **324**, 85 (2009).
[94] P. S. Gromski, A. B. Henson, J. M. Granda, and L. Cronin, "How to explore chemical space using algorithms and automation," Nat. Rev. Chem. **3**, 119 (2019).
[95] J. P. Zwolak and J. M. Taylor, in Colloquium: Advances in automation of Quantum Dot Devices Control, 2021.
[96] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "NVIDIA tesla: A unified graphics and computing architecture," IEEE Micro **28**, 39 (2008).

04 September 2026 20:03:04

© Author(s) 2024

[97]S. Balensiefer, L. Kregor-Stickles, and M. Oskin, "An evaluation framework and instruction set architecture for ion-trap based quantum micro-architectures," in *32nd International Symposium on Computer Architecture (ISCA'05)* (IEEE, 2005), pp. 186–196.
[98]R. S. Smith, M. J. Curtis, and W. J. Zeng, "A practical quantum instruction set architecture," arXiv:1608.03355 [quant-ph] (2017).
[99]X. Fu, L. Riesebos, M. Rol, J. Van Straten, J. Van Someren, N. Khammassi, I. Ashraf, R. Vermeulen, V. Newsum, K. Loh *et al.*, "eQASM: An executable quantum instruction set architecture," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (IEEE, 2019), pp. 224–237.
[100]A. Butko, G. Michelogiannakis, S. Williams, C. Iancu, D. Donofrio, J. Shalf, J. Carter, and I. Siddiqi, "Understanding quantum control processor capabilities and limitations through circuit characterization," in *2020 International Conference on Rebooting Computing (ICRC)* (IEEE Computer Society, Los Alamitos, CA, USA, 2020), pp. 66–75.
[101]A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanović, "The RISC-V instruction set manual volume II: Privileged architecture version 1.9," Tech. Rep. UCB/EECS-2016-129, EECS Department, University of California, Berkeley, 2016.
[102]K. Asanović, R. Avižienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, P. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo, and A. Waterman, "The rocket chip generator," Tech. Rep. UCB/EECS-2016-17, EECS Department, University of California, 2016.
[103]A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," IEEE Micro **40**, 10 (2020).
[104]D. Lobser, J. Goldberg, A. J. Landahl, P. Maunz, B. C. A. Morrison, K. Rudinger, A. Russo, B. Ruzic, D. Stick, J. V. D. Wall, and S. M. Clark, "JaqalPaw: A guide to defining pulses and waveforms for Jaqal," arXiv:2305.02311 [quant-ph] (2023).
[105]R. B.-S. Tsai, H. Silvério, and L. Henriet, "Pulse-level scheduling of quantum circuits for neutral-atom devices," arXiv:2206.05144 [quant-ph] (2022).
[106]Y. Xu, G. Huang, D. I. Santiago, and I. Siddiqi, "Radio frequency mixing modules for superconducting qubit room temperature control systems," Rev. Sci. Instrum. **92**, 075108 (2021).
[107]N. Fruitwala, G. Huang, Y. Xu, R. Naik, K. Nowrouzi, D. Santiago, and I. Siddiqi, *Distributed Processor for FPGA-Based Superconducting Qubit Control Using Qubic* (Bulletin of the American Physical Society, 2023).
[108]Y. Xu, G. Huang, N. Fruitwala, A. Rajagopala, R. K. Naik, K. Nowrouzi, D. I. Santiago, and I. Siddiqi, "QubiC 2.0: An extensible open-source qubit control system capable of mid-circuit measurement and feed-forward," arXiv:2309.10333 (2023).
[109]A. Butko, Y. Xu, G. Huang, R. Naik, D. Santiago, and I. Siddiqi, *Real-time Fast Feedback Experiment Enabled by a Customized FPGA-Based Control System* (Bulletin of the American Physical Society, 2023).
[110]H. Johnson, S. Zorzetti, and J. Saniie, "Exploration of optimizing FPGA-based qubit controller for experiments on superconducting quantum computing hardware," arXiv:2305.06976 (2023).
[111]A. V. Dixit, S. Chakram, K. He, A. Agrawal, R. K. Naik, D. I. Schuster, and A. Chou, "Searching for dark matter with a superconducting qubit," Phys. Rev. Lett. **126**, 141302 (2021).
[112]AMD-Xilinx, *ZCU111 evaluation board user guide (UG1271)*, 2023.
[113]AMD-Xilinx, *ZCU216 evaluation board user guide (UG1390)*, 2020.
[114]A. Browaeys and T. Lahaye, "Many-body physics with individually controlled Rydberg atoms," Nat. Phys. **16**, 132 (2020).
[115]L. Henriet, L. Beguin, A. Signoles, T. Lahaye, A. Browaeys, G.-O. Reymond, and C. Jurczak, "Quantum computing with neutral atoms," Quantum **4**, 327 (2020).
[116]M.-T. Nguyen, J.-G. Liu, J. Wurtz, M. D. Lukin, S.-T. Wang, and H. Pichler, "Quantum optimization with arbitrary connectivity using Rydberg atom arrays," PRX Quantum **4**, 010316 (2023).
[117]J. Werschnik and E. Gross, "Quantum optimal control theory," J. Phys. B: At., Mol. Opt. Phys. **40**, R175 (2007).
[118]C. P. Koch, U. Boscain, T. Calarco, G. Dirr, S. Filipp, S. J. Glaser, R. Kosloff, S. Montangero, T. Schulte-Herbrüggen, D. Sugny, and F. K. Wilhelm, "Quantum optimal control in quantum technologies. Strategic report on current status, visions and goals for research in europe," EPJ Quantum Technol. **9**, 19 (2022).
[119]M. Goerz, D. Basilewitsch, F. Gago-Encinas, M. G. Krauss, K. P. Horn, D. M. Reich, and C. Koch, "Krotov: A Python implementation of Krotov's method for quantum optimal control," SciPost Phys. **7**, 080 (2019).
[120]J. Kelly, R. Barends, B. Campbell, Y. Chen, Z. Chen, B. Chiaro, A. Dunsworth, A. G. Fowler, I.-C. Hoi, E. Jeffrey, A. Megrant, J. Mutus, C. Neill, P. J. J. O'Malley, C. Quintana, P. Roushan, D. Sank, A. Vainsencher, J. Wenner, T. C. White, A. N. Cleland, and J. M. Martinis, "Optimal quantum control using randomized benchmarking," Phys. Rev. Lett. **112**, 240504 (2014).
[121]M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (USENIX Association, Savannah, GA, 2016), pp. 265–283.
[122]M. Rossignolo, T. Reisser, A. Marshall, P. Rembold, A. Pagano, P. J. Vetter, R. S. Said, M. M. Müller, F. Motzoi, T. Calarco, F. Jelezko, and S. Montangero, "QuOCS: The quantum optimal control suite," Comput. Phys. Commun. **291**, 108782 (2023).
[123]T. Propson, B. E. Jackson, J. Koch, Z. Manchester, and D. I. Schuster, "Robust quantum optimal control with trajectory optimization," Phys. Rev. Appl. **17**, 014036 (2022).
[124]C. Gidney, "Stim: A fast stabilizer circuit simulator," Quantum **5**, 497 (2021).
[125]See https://github.com/qc-design/plaquette for more information about Q. Design, Plaquette, 2023.
[126]See https://github.com/abdullahkhalids/stac for more information about A. Khalid, Stac, 2023.
[127]O. Higgott, "PyMatching: A python package for decoding quantum codes with minimum-weight perfect matching," arXiv:2105.13082 (2021).
[128]O. Higgott and C. Gidney, "Sparse blossom: Correcting a million errors per core second with minimum-weight matching," arXiv:2303.15933 (2023).
[129]M. J. Reagor, T. C. Bohdanowicz, D. R. Perez, E. A. Sete, and W. J. Zeng, "Hardware optimized parity check gates for superconducting surface codes," arXiv:2211.06382 (2022).
[130]J. Sáenz, J. Chacón, L. De La Torre, A. Visioli, and S. Dormido, "Open and low-cost virtual and remote labs on control engineering," IEEE Access **3**, 805 (2015).
[131]L. Gomes and S. Bogosyan, "Current trends in remote laboratories," IEEE Trans. Ind. Electron. **56**, 4744 (2009).
[132]C. Kourkoumelis and S. Vourakis, "Hypatia—An online tool for atlas event visualization," Phys. Educ. **49**, 21 (2014).
[133]J. C. Norman, D. Jung, Y. Wan, and J. E. Bowers, "Perspective: The future of quantum dot photonic integrated circuits," APL Photonics **3**, 030901 (2018).
[134]J. L. DuBois, V. Lordi, Y. J. Rosen, and X. Wu, "Request for the establishment of quantum information foundries," Technical Report 10.2172/1670543, 2020.
[135]E. Bussmann, R. E. Butera, J. H. G. Owen, J. N. Randall, S. M. Rinaldi, A. D. Baczewski, and S. Misra, "Atomic-precision advanced manufacturing for Si quantum computing," MRS Bull. **46**, 607 (2021).
[136]E. Altman, K. R. Brown, G. Carleo, L. D. Carr, E. Demler, C. Chin, B. DeMarco, S. E. Economou, M. A. Eriksson, K.-M. C. Fu, M. Greiner, K. R. Hazzard, R. G. Hulet, A. J. Kollár, B. L. Lev, M. D. Lukin, R. Ma, X. Mi, S. Misra, C. Monroe, K. Murch, Z. Nazario, K.-K. Ni, A. C. Potter, P. Roushan, M. Saffman, M. Schleier-Smith, I. Siddiqi, R. Simmonds, M. Singh, I. Spielman, K. Temme, D. S. Weiss, J. Vučković, V. Vuletić, J. Ye, and M. Zwierlein, "Quantum simulators: Architectures and opportunities," PRX Quantum **2**, 017003 (2021).
[137]D. Awschalom, K. K. Berggren, H. Bernien, S. Bhave, L. D. Carr, P. Davids, S. E. Economou, D. Englund, A. Faraon, M. Fejer, S. Guha, M. V. Gustafsson, E. Hu, L. Jiang, J. Kim, B. Korzh, P. Kumar, P. G. Kwiat, M. Lončar, M. D. Lukin, D. A. Miller, C. Monroe, S. W. Nam, P. Narang, J. S. Orcutt, M. G. Raymer, A. H. Safavi-Naeini, M. Spiropulu, K. Srinivasan, S. Sun, J. Vučković, E. Waks, R. Walsworth, A. M. Weiner, and Z. Zhang, "Development of quantum interconnects (QuICs) for next-generation information technologies," PRX Quantum **2**, 017002 (2021).

04 September 2026 20:03:04

© Author(s) 2024

[138]Y. Alexeev, D. Bacon, K. R. Brown, R. Calderbank, L. D. Carr, F. T. Chong, B. DeMarco, D. Englund, E. Farhi, B. Fefferman, A. V. Gorshkov, A. Houck, J. Kim, S. Kimmel, M. Lange, S. Lloyd, M. D. Lukin, D. Maslov, P. Maunz, C. Monroe, J. Preskill, M. Roetteler, M. J. Savage, and J. Thompson, "Quantum computer systems for scientific discovery," PRX Quantum **2**, 017001 (2021).
[139]V. Frey, R. Rademacher, E. Durso-Sabina, N. Greenberg, N. Videnov, M. L. Day, R. Islam, and C. Senko, "Programming the full stack of an open-access quantum computer," arXiv:2106.06549 [quant-ph] (2021).
[140]A. Solfanelli, A. Santini, and M. Campisi, "Experimental verification of fluctuation relations with a quantum computer," PRX Quantum **2**, 030353 (2021).
[141]X. Mi, M. Ippoliti, C. Quintana, A. Greene, Z. Chen, J. Gross, F. Arute, K. Arya, J. Atalaya, R. Babbush *et al.*, "Time-crystalline eigenstate order on a quantum processor," Nature **601**, 531 (2022).
[142]D. Liukkonen, Cloud queue for quantum devices, 2023.
[143]Q. Flagship, The quantum future begins in prague, 2023.
[144]I. UK, SBRI: Development and delivery of quantum computing testbeds, 2023.
[145]B. C. A. Morrison, A. J. Landahl, D. S. Lobser, K. M. Rudinger, A. E. Russo, J. W. Van Der Wall, and P. Maunz, "Just another quantum assembly language (Jaqal)," in *2020 IEEE Int. Conf. Quant. Comp. Eng. (QCE)* (IEEE, 2020), pp. 402–408.
[146]PsiQuantum, PsiQuantum and GLOBALFOUNDRIES to Build the World's First Full-scale Quantum Computer, 2022.
[147]Xanadu, Xanadu announces collaboration with GlobalFoundries to accelerate development of fault-tolerant photonic quantum computers, 2022.
[148]J. Hsu, Rigetti Launches Full-Stack Quantum Computing Service and Quantum IC Fab, 2017.
[149]UCSB, UCSB NSF Quantum Foundry, 2022.
[150]MonArk, MonArk NSF Quantum Foundry, 2022.
[151]LPS, LPS Qubit Collaboratory, 2022.
[152]Quantware, Quantware, 2022.
[153]A. Cross, A. Javadi-Abhari, T. Alexander, N. De Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, and B. R. Johnson, "OpenQASM 3: A broader and deeper quantum Assembly Language," ACM Trans. Quantum Comput. **3**, 1 (2022).
[154]A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," arXiv:1707.03429 [quant-ph] (2017).
[155]N. Mohseni, P. L. McMahon, and T. Byrnes, "Ising machines as hardware solvers of combinatorial optimization problems," Nat. Rev. Phys. **4**, 363 (2022).
[156]A. M. C. Dawes, "Undergraduate quantum mechanics: A numerical approach using QuTiP," arXiv:1909.13651 [physics.ed-ph] (2019).
[157]F. Bouquet, J. Bobroff, M. Fuchs-Gallezot, and L. Maurines, "Project-based physics labs using low-cost open-source hardware," Am. J. Phys. **85**, 216 (2017).
[158]Z. Gingl, J. Mellár, T. Szépe, G. Makan, R. Mingesz, G. Vadai, and K. Kopasz, "Universal Arduino-based experimenting system to support teaching of natural sciences," J. Phys.: Conf. Ser. **1287**, 012052 (2019).
[159]N. Shammah, S. Ahmed, S. Kaiser, and A. Panigrahi, and tiagob01, *Make your code count*, 2023.

04 September 2026 20:03:04

© Author(s) 2024